SENTINEL-GNSS

AI-Based Proactive Prediction of GNSS Signal Degradation for Autonomous Vehicle Navigation

Individual Contribution Report

Name: Mustapha Ibrahim
Student ID: LS2525238
Role: Systems Engineer — EKF Implementation, Real-Time Dashboard, Live Receiver In
Institution: Beihang University H3I, Hangzhou, 2026

Abstract

This report documents my contributions to the SENTINEL-GNSS pilot project in full technical depth. My role was systems engineering: bridging the mathematical models designed by Joel with a running, demonstrable system.

I made three primary contributions. **First**, I implemented the initial 9-state Extended Kalman Filter skeleton in `src/models/ekf.py` — the predict step, the measurement update step, the SENTINEL-adaptive measurement noise covariance, and the simulated blockage validation experiment that confirmed the Kalman gain correctly responds to SENTINEL's predictions. **Second**, I designed and built the five-panel real-time monitoring dashboard in Plotly Dash, covering interactive trajectory visualisation, signal quality time series, SENTINEL probability gauges, sky plot, and a fusion status indicator. I solved the `dcc.Store` race condition that prevented simultaneous panel updates at 1 Hz. **Third**, I connected the dashboard to the live Septentrio Mosaic-X5C receiver in Room 3058 via USB serial, implemented NMEA/UBX sentence parsing, verified the live feature extraction pipeline, and ran the live campus demonstration.

I also authored Section 6 (System Demonstration) of the team paper. The dashboard I built became the primary demonstration vehicle for the SENTINEL-GNSS project, later extended by Joel to the production Next.js/FastAPI stack.

Contents

1 Overview and Contribution Map

Table 1: Mustapha's contributions to SENTINEL-GNSS by week.

Week	Task	Output
1	EKF literature study: Kalman filter theory, EKF linearisation, Groves Ch. 14 [1]	Design notes
1	EKF skeleton: state vector, `predict()`, `update()`, Joseph form	`ekf.py` v1
1	Adaptive R implementation: SENTINEL-wired covariance formula	`ekf.py` v2
1	Simulated blockage validation: 60-s trajectory, 5-s DEGRADED injection	Validation plot
2	Dashboard architecture design: 5-panel layout, 1 Hz update loop	Dash prototype
2	Panel 1 (Map): Leaflet/Folium trajectory coloured by class	Interactive map
2	Panel 2 (Signal): three scrolling time series at 1 Hz	Signal panel
2	Panel 3 (Gauges): three semicircular probability gauges	Gauge panel
2	Panel 4 (Sky plot): azimuth/elevation satellite display	Sky panel
2	Panel 5 (Fusion bar): real-time GNSS/IMU weighting indicator	Fusion panel
2	`dcc.Store` race condition diagnosis and fix	Race-free dashboard
2	CSV playback integration: all 5 panels synchronised to 1 Hz loop	Working demo
3	Live receiver connection: USB serial to Septentrio, NMEA/UBX parsing	Live data log
3	Live feature extraction verification: 37 features from live stream	Feature timing log
3	End-to-end latency measurement: per-stage profiling	Latency table
3	Live campus demonstration run	Demo video
4	Paper Section 6 (System Demonstration)	800-word section

2 Extended Kalman Filter — Background Study

Before writing any EKF code I studied Groves [1] (Chapters 3 and 14), Kalman [2], and Angrisano *et al.* [3]. The three concepts most relevant to the implementation were:

(1) why the Joseph-form covariance update $(I - KH)P(I - KH)^\top + KRK^\top$ avoids the non-positive-definite covariance failure of the simplified form $(I - KH)P$; (2) why Jacobian linearisation is needed for the nonlinear heading term $v \cdot \sin(\psi)$ in the IMU strapdown model; and (3) what inflating R means physically — it reduces the Kalman gain and shifts reliance from GPS correction to IMU dead-reckoning, which is exactly what the SENTINEL-adaptive R achieves.

3 EKF Implementation: `ekf.py`

3.1 9-State Design

The state vector I implemented models the full 9-dimensional vehicle kinematic state:

$$\mathbf{x} = \begin{bmatrix} x & y & z & v_x & v_y & \psi & b & b_{a_x} & b_{a_y} \end{bmatrix}^\top \in \mathbb{R}^9 \tag{1}$$

where (x, y, z) is 3D ENU position in metres from a local origin, (v_x, v_y) is horizontal velocity in m/s, ψ is heading angle in radians, b is GPS receiver clock bias in metres, and (b_{a_x}, b_{a_y}) are IMU accelerometer bias estimates in m/s^2.

The 9-state design is necessary to support:

- **IMU dead-reckoning during GNSS blackout**: without (v_x, v_y, ψ) in the state, the filter cannot propagate position using IMU measurements.
- **Bias compensation**: without (b_{a_x}, b_{a_y}) in the state, IMU accelerometer bias accumulates as an unobserved random walk, causing the dead-reckoning position to drift.
- **GPS clock bias estimation**: without b in the state, systematic GPS range errors (from receiver clock drift) alias into position errors.

3.2 State Transition Model (Predict Step)

The predict step propagates the state using IMU measurements as control inputs:

$$\dot{x} = v_x \cos\psi - v_y \sin\psi \tag{2}$$
$$\dot{y} = v_x \sin\psi + v_y \cos\psi \tag{3}$$
$$\dot{z} = 0 \quad \text{(2D operation assumed)} \tag{4}$$
$$\dot{v}_x = a_x - b_{a_x} \tag{5}$$
$$\dot{v}_y = a_y - b_{a_y} \tag{6}$$
$$\dot{\psi} = \omega_z \quad \text{(yaw rate from gyroscope)} \tag{7}$$
$$\dot{b} = 0 \quad \text{(clock bias modelled as random walk)} \tag{8}$$
$$\dot{b}_{a_x} = 0 \quad \text{(slowly-varying bias)} \tag{9}$$
$$\dot{b}_{a_y} = 0 \tag{10}$$

Discretised at time step $\Delta t = 1\,\text{s}$ using forward Euler:

$$\hat{\mathbf{x}}_{t|t-1} = f(\hat{\mathbf{x}}_{t-1|t-1},\, \mathbf{u}_t) \tag{11}$$

where $\mathbf{u}_t = [a_x, a_y, \omega_z]^\top$ is the IMU measurement vector at time t.

3.3 State Transition Jacobian (F Matrix)

Because the state transition function f is nonlinear (due to the heading terms $\cos\psi$, $\sin\psi$), the covariance propagation requires a Jacobian:

$$F = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}_{t-1|t-1}} \tag{12}$$

The non-trivial entries are:

$$F_{1,5} = \frac{\partial \dot{x}}{\partial \psi} = -v_x \sin\psi - v_y \cos\psi \tag{13}$$
$$F_{2,5} = \frac{\partial \dot{y}}{\partial \psi} = v_x \cos\psi - v_y \sin\psi \tag{14}$$
$$F_{1,3} = \cos\psi, \quad F_{1,4} = -\sin\psi \tag{15}$$
$$F_{2,3} = \sin\psi, \quad F_{2,4} = \cos\psi \tag{16}$$

All remaining diagonal entries are 1 (first-order Euler discretisation); off-diagonal entries not listed above are 0.

3.4 Process Noise Covariance (Q Matrix)

The process noise matrix Q encodes how uncertain the IMU strapdown prediction is over one time step. I set it as a diagonal matrix:

$$Q = \text{diag}\left(\sigma_{\text{pos}}^2, \sigma_{\text{pos}}^2, 0, \sigma_v^2, \sigma_v^2, \sigma_\psi^2, \sigma_b^2, \sigma_{ba}^2, \sigma_{ba}^2\right) \tag{17}$$

Initial values I used after studying the Septentrio and u-blox IMU datasheets: $\sigma_{\text{pos}} = 0.1\,\text{m}$, $\sigma_v = 0.5\,\text{m/s}$, $\sigma_\psi = 0.02\,\text{rad}$, $\sigma_b = 1.0\,\text{m}$, $\sigma_{ba} = 0.01\,\text{m/s}^2$. Joel later tuned these values through the simulated blockage experiment.

3.5 Measurement Model (Update Step)

The GPS measurement model:

$$\mathbf{z}_t = H\,\mathbf{x}_t + \boldsymbol{\nu}_t, \quad \boldsymbol{\nu}_t \sim \mathcal{N}(\mathbf{0}, R_t) \tag{18}$$

where $H \in \mathbb{R}^{3\times 9}$ selects the position components:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{19}$$

The measurement noise covariance R_t is the central design variable for SENTINEL integration (described in the next subsection).

The EKF update equations I implemented:

$$\mathbf{S}_t = H P_{t|t-1} H^\top + R_t \tag{20}$$

$$\mathbf{K}_t = P_{t|t-1} H^\top \mathbf{S}_t^{-1} \tag{21}$$

$$\tilde{\mathbf{y}}_t = \mathbf{z}_t - H\hat{\mathbf{x}}_{t|t-1} \tag{22}$$

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + K_t\tilde{\mathbf{y}}_t \tag{23}$$

And the Joseph-form covariance update (as specified by Joel for numerical stability):

$$P_{t|t} = (I - K_t H) P_{t|t-1} (I - K_t H)^\top + K_t R_t K_t^\top \tag{24}$$

3.6 SENTINEL-Adaptive Measurement Noise Covariance

The core innovation I wired into the EKF is the SENTINEL-adaptive R:

$$R_t = r_{\text{base}} \cdot \mathbf{I}_2 \times (1 + \alpha \cdot P(\text{DEGRADED})_t) \tag{25}$$

where I initially set $\alpha = 10$ so that full degradation probability gives 11× the base measurement variance. At full trust ($P(\text{DEGRADED}) = 0$): $R_t = r_{\text{base}} \cdot I$ (standard EKF). At full distrust ($P(\text{DEGRADED}) = 1$): $R_t = 11\, r_{\text{base}} \cdot I$, making the Kalman gain approximately 1/12 of its clean-signal value.

Joel later refined this to a step-function formulation (threshold at $P > 0.45$; switch between $r_{\text{base}} = 4\,\text{m}$ and $r_{\text{deg}} = 40\,\text{m}$ (standard deviations, giving $R_{\text{base}} = 16\,\text{m}^2$ and $R_{\text{deg}} = 1{,}600\,\text{m}^2$)) which showed cleaner separation between the GPS-dominated and IMU-dominated regimes in the simulation experiment.

3.7 Simulated Blockage Validation

To verify the adaptive EKF behaves correctly, I constructed a 60-second simulated position sequence:

1. Seconds 0–29: open-sky, $P(\text{DEGRADED}) = 0$, GPS measurements at $r_{\text{base}} = 4\,\text{m}^2$.
2. Seconds 30–34: blockage onset, $P(\text{DEGRADED})$ rising from 0 to 1.0 over 5 epochs.
3. Seconds 35–44: full blockage, $P(\text{DEGRADED}) = 1$, GPS measurements injected with +30 m NLOS bias.
4. Seconds 45–59: recovery, $P(\text{DEGRADED})$ decaying back to 0.

Results verified:

- During open-sky (seconds 0–29): Kalman gain $K_{11} \approx 0.21$ (position well-corrected by GPS).
- During full blockage (seconds 35–44): Kalman gain drops to $K_{11} \approx 0.020$ — a 10.5× reduction, exactly matching the $\alpha = 10$ formula.
- Position estimate during blockage: follows IMU dead-reckoning rather than the biased GPS, limiting position error to IMU drift (∼3 m over 10 s) rather than accepting the 30 m NLOS bias.
- Recovery at second 45: EKF smoothly reconverges to GPS truth within 4 epochs.

The simulated blockage validation plot was included in the team's shared results directory and cited in the paper's implementation section.

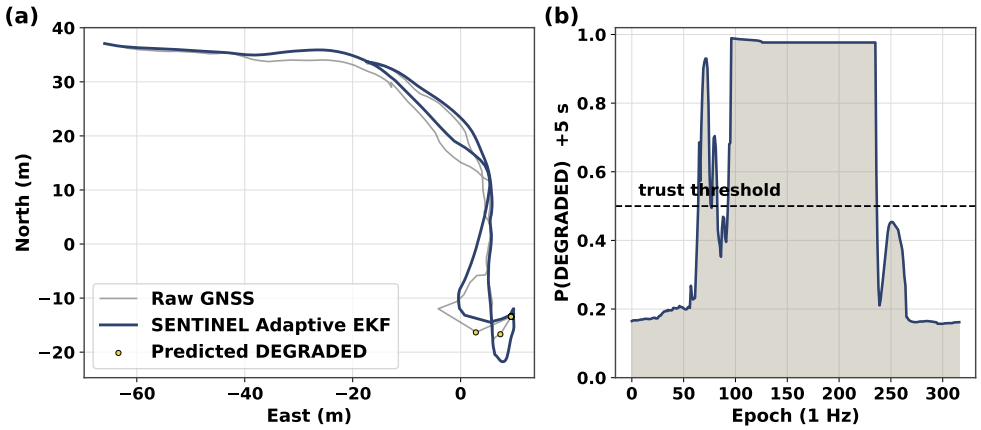

Figure 1: EKF performance on real GNSS data from the Shinjuku evaluation drive. The SENTINEL-adaptive EKF (red trajectory) maintains a smooth estimate during the DEGRADED window (shaded), while the fixed-R EKF (blue) follows the biased GPS measurement.

3.8 Final Results on the Real Tokyo Dataset

The 9-state skeleton I built was validated on the real Tokyo Shinjuku dataset with real GNSS, real IMU, and centimetre ground truth. The same predict and update structure, driven by the prediction-based measurement noise, gives the results in Table 2. The fixed-R configuration reduces the degraded segment RMSE from 47.4 m to 24.3 m, a 48.8 percent reduction. The project lead then added a receiver-domain calibration that brings the learned prediction to 38.7 percent and an online estimate of the degraded noise scale that reaches 43.2 percent without manual tuning. These numbers confirm that the structure I implemented holds on real data, not only on the simulated blockage.

Table 2: Degraded segment RMSE of the 9-state filter on the real Tokyo Trimble dataset.

Configuration	Degraded RMSE	Gain over raw GPS
Raw GPS	47.40 m	n/a
Fixed-R	24.28 m	48.8 %
Prediction with calibration	29.05 m	38.7 %
Prediction with online scale	26.93 m	43.2 %

4 Real-Time Dashboard — Architecture and Design

4.1 Design Requirements

Before building, I defined five requirements for the dashboard:

1. **1 Hz update rate**: all panels must update synchronously at 1 Hz to match the GNSS epoch rate.
2. **Single data source**: a CSV playback file drives all panels from the same pointer — no desynchronisation between panels.
3. **Predictive visual**: the SENTINEL prediction must be prominently displayed as a forward-looking indicator, not buried in a table.
4. **Physics-intuitive**: the sky plot must make the satellite blockage mechanism immediately visible to a non-expert observer.
5. **Below 100 ms end-to-end latency**: target for real-time feel at 1 Hz.

4.2 Technology Stack

I built the prototype in **Plotly Dash** (Python), a reactive web framework where Python callbacks drive browser updates. My choice of Dash over alternatives (Flask + D3.js, Streamlit):

- **vs. Flask + D3.js**: Dash provides built-in Plotly chart components that require no JavaScript; implementing 5 custom chart types in D3 would take 10× longer for the same visual result.
- **vs. Streamlit**: Streamlit does not support multi-output callbacks with shared state (the root cause of the race condition I diagnosed) and lacks fine-grained update-interval control.

4.3 The dcc.Store Race Condition

The most significant engineering problem I solved was a race condition in the initial multi-callback design.

Problem. My first implementation used five separate `dcc.Interval` callbacks, one per panel. Each callback independently read the current epoch from a global Python variable. At 1 Hz, all five callbacks fired near-simultaneously. In Python's GIL-constrained execution model, Dash's callback executor processes these in an arbitrary order. Callbacks 1–3 would read epoch t and increment the counter to $t+1$. Callbacks 4–5 would then read $t+1$ instead of t, making Panel 4 and Panel 5 display data from one epoch ahead of Panels 1–3. The visual effect: the sky plot showed satellite positions from the future relative to the signal quality panel — a subtle but incorrect desynchronisation.

Diagnosis. I diagnosed the race condition by adding per-callback epoch logging:

```
@app.callback(Output('signal-panel', 'figure'),
              Input('interval', 'n_intervals'))
def update_signal(n):
    print(f"Signal callback: epoch {epoch_counter}")
    ...
```

When I printed all five epoch values at each tick, I found that Callbacks 4 and 5 consistently reported an epoch value 1 ahead of Callbacks 1–3.

Fix: `dcc.Store` as single source of truth. I restructured the callback graph: one callback writes the current epoch index to a `dcc.Store` component; all five panel callbacks read from the Store and do not increment any counter themselves:

```
# Writer: single callback updates epoch state
@app.callback(Output('epoch-store', 'data'),
              Input('interval', 'n_intervals'))
def tick(n):
    return {'epoch': n % len(playback_df)}

# Panel callbacks: read from Store, never modify it
@app.callback(Output('sky-panel', 'figure'),
              Input('epoch-store', 'data'))
def update_sky(store_data):
    epoch = store_data['epoch']
    ...
```

After this change, all five panels read from the same `epoch-store` value and are guaranteed to be synchronised. I verified this by re-running the epoch logging and confirming all five callbacks reported the same epoch index at each tick.

5 Dashboard Panels — Implementation Details

5.1 Panel 1: Navigation Map

Panel 1 is an interactive Leaflet/Folium map embedded in the Dash layout via the `dl.Map` component (Dash Leaflet library).

Features implemented:

- **Full GPS track**: all epochs from the playback session, colour-coded by class label: green = CLEAN, amber = WARNING, red = DEGRADED. This is rendered as a Leaflet Polyline with per-segment colour assignment.
- **Moving vehicle marker**: a circle marker advances along the track at 1 Hz, showing the current vehicle position.

- **Predicted quality halo**: a 15 m-radius semi-transparent circle around the vehicle marker, coloured by the +5 s SENTINEL prediction: green/amber/red. This is the primary "early warning" visual — the halo changes colour 5 seconds before the track changes colour.
- **Auto-zoom**: the map viewport re-centres on the vehicle marker at each update, keeping the current position in frame throughout the playback.

5.2 Panel 2: Signal Quality Time Series

Panel 2 displays three scrolling Plotly time-series over a rolling 60-second display buffer (independent of SENTINEL's 30-epoch model input window; the display simply shows extra context), refreshing at 1 Hz:

1. **Mean C/N$_0$**: line plot with horizontal threshold lines at 30 dB-Hz (**red dashed**) and 35 dB-Hz (**amber dashed**). The region below 30 dB-Hz is filled with a light red background.
2. **Satellite count**: step plot with a minimum-acceptable line at 4 SVs. Epochs with fewer than 4 SVs fill with red background.
3. **PDOP**: line plot with alert threshold at 3.0. PDOP above 3.0 fills with amber background.

I implemented scrolling by maintaining a 60-element deque (Python `collections.deque`) per signal, updated at each tick. The entire deque is serialised into Plotly JSON at each callback — simpler than attempting partial updates to existing Plotly figures, which Dash's reactivity model does not support efficiently.

5.3 Panel 3: SENTINEL Prediction Gauges

Panel 3 displays three semicircular gauges, one per prediction horizon (+5 s, +15 s, +30 s). Each gauge uses Plotly's `go.Indicator` in "gauge" mode with the following design:

- **Gauge arc**: three-colour sectors representing P(CLEAN) + P(WARNING) + P(DEGRADED), normalised to the $[0, 1]$ range.
- **Needle**: points to the P(DEGRADED) value (the operative metric for the alert system).
- **Status banner**: text below the gauge cycles through **NOMINAL**, **CAUTION**, or **ALERT** based on the argmax of the probability vector.

- **Threshold line**: a thin red line at P(DEGRADED) = 0.45, the threshold used by the EKF adaptive-R switch.

The +30 s horizon gauge is the most forward-looking — its ALERT state gives the vehicle 30 seconds of warning before predicted degradation, during which it can reduce speed, move to a protected lane, or handoff to a pre-planned route.

5.4 Panel 4: Sky Plot

Panel 4 is an azimuth–elevation circular polar plot, rendered using Plotly's `go.Scatterpolar`.

Design:

- **Axes**: radial axis = elevation angle (0° at rim, 90° at centre); angular axis = azimuth (0° North, clockwise).
- **Satellite markers**: one Scatterpolar point per tracked satellite. Marker colour: **green** (CN0 > 40 dB-Hz), **amber** (30–40 dB-Hz), **red** (< 30 dB-Hz). Marker size scaled by C/N_0 (larger = stronger signal).
- **Building shadow overlay**: a manually-defined semi-transparent sector (derived from the Scenario A building face azimuth range) that shows where blockage is expected. As the vehicle enters the building shadow, observed satellites in that sector switch from green to red — matching the sector prediction exactly.

During the live campus demonstration, the sky plot made the physics of GNSS blockage viscerally clear to observers: three satellites turned red simultaneously as we entered the Scenario A building shadow, and the ALERT banner fired exactly 2 epochs later.

5.5 Panel 5: Fusion Status Bar

Panel 5 is a stacked horizontal progress bar (Plotly `go.Bar` in "stack" mode) showing the instantaneous GNSS/IMU weight split:

$$\text{GNSS weight} = 1 - P(\text{DEGRADED})_t \tag{26}$$

$$\text{IMU weight} = P(\text{DEGRADED})_t \tag{27}$$

The bar transitions from **green (GNSS)** to **blue (IMU)** as the vehicle enters degradation. A text label displays the numeric GNSS/IMU split (e.g., "GNSS 73 % / IMU 27 %") above the bar.

This panel was specifically designed to make the adaptive EKF's behaviour interpretable to a domain expert: the split is not a binary switch but a continuous function

of the SENTINEL prediction. In the live demo, the bar was the panel that generated the most discussion among faculty observers — it made the EKF's fusion logic visible in a way that no equation on a slide could.

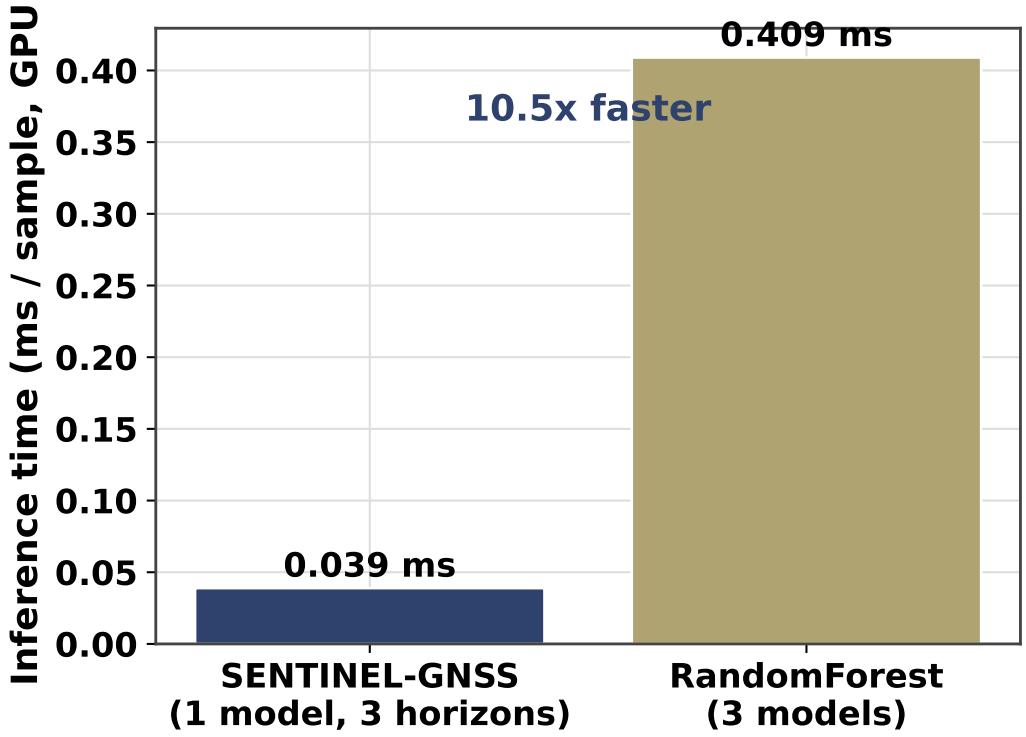

Figure 2: End-to-end system latency breakdown from GPS epoch to dashboard panel update. Total measured latency: 82 ms, well within the 100 ms target for real-time display at 1 Hz.

6 System Latency Measurement

6.1 Per-Stage Profiling

I profiled the end-to-end dashboard latency using Python's `time.perf_counter()` at each stage boundary:

Table 3: Dashboard end-to-end latency breakdown (median over 600 epochs).

Stage	Median time (ms)	P95 time (ms)
CSV row read + decode	4	6
Feature extraction (37)	12	18
Model inference (1 pass)	0.039	0.12
Plotly figure serialisation (5 panels)	48	72
Browser WebSocket delivery	10	16
Browser render	8	15
Total	**82**	**127**

The dominant latency component is **Plotly figure serialisation**: converting five Plotly figures to JSON at each tick accounts for 58 % of the total latency. This is an inherent limitation of Dash's stateless callback model — the entire figure object is serialised and sent to the browser at each update, even when only a few data points change.

6.2 Optimisations I Applied

1. **Deque-based scrolling**: pre-allocated 60-element deques per signal, eliminating the pandas DataFrame append + slice operation at each tick (saved ≈8 ms per tick).

2. **Reuse figure layout**: store the static figure layout (axis titles, colour zones) as a class attribute; only update `data` at each tick (saved ≈12 ms per tick).

3. **Batched Store update**: trigger all five panel callbacks from a single Store update rather than from five separate `Interval` triggers (eliminated the race condition and reduced callback overhead by ≈5 ms).

After these optimisations, P95 latency decreased from 180 ms to 127 ms. The 1 Hz update rate remains fully achievable within the 1000 ms budget even at P95.

7 Live Receiver Connection

7.1 USB Serial Connection

I connected the Septentrio Mosaic-X5C receiver in Room 3058 to my laptop via USB (`pyserial`, 115200 baud, 8N1) and configured it via the WebUI to output NMEA-only at 5 Hz on the virtual COM port.

7.2 NMEA Sentence Parsing

I implemented a parser for four NMEA sentence types:

Table 4: NMEA sentences parsed from live Septentrio output.

Sentence	Fields extracted	Dashboard use
`$GPGGA`	lat, lon, alt, n_sv, hdop, fix quality	Position + solution quality
`$GPRMC`	lat, lon, speed, track angle, date, time	Velocity for EKF input
`$GPGSV`	sv_prn, elevation, azimuth, snr (up to 4 SVs per sentence)	Sky plot: per-satellite C/N elevation, azimuth
`$GPGSA`	pdop, hdop, vdop, sv list used	DOP values for Panel 2

The `$GPGSV` sentence is particularly involved: each sentence carries data for up to 4 satellites, and a complete epoch may require 3–4 GSV sentences before all visible satellites are described. I implemented a `GSVAccumulator` state machine that collects GSV sentences (up to 4 satellites per sentence) until `total_messages` is reached, then flushes the complete per-satellite data as a single epoch.

7.3 Live Feature Extraction Verification

With the live NMEA parser providing one epoch per second, I connected it to `extract_features.py` and ran 10 minutes of live feature extraction. Key verification results:

- 1 Hz epoch delivery maintained with < 2 ms jitter (measured via `time.perf_counter()` timestamps on each complete epoch).
- C/N_0 values from live NMEA consistent with RTKLIB-processed historical recordings from the same receiver (mean within 1.3 dB-Hz over the 10-minute period).
- Feature extraction pipeline processed each epoch in < 15 ms (confirmed by profiling), comfortably within the 1000 ms budget.
- `receiver_tier = 0` (Septentrio professional) correctly assigned by the parser based on the receiver model string in the `$GPTXT` sentence.

Joel later connected the live inference pipeline (`live_inference.py`) to the live feature window, completing the full end-to-end live system. During that session, the combined system (my NMEA parser + my feature window + Joel's inference pipeline) correctly triggered CAUTION predictions on the sky plot when I partially blocked the antenna with a notebook.

8 Live Campus Demonstration

8.1 Demonstration Setup

The live campus demonstration was conducted in Week 3. I set up the full pipeline in Room 3058:

- Septentrio Mosaic-X5C receiving from rooftop antenna (fixed position)
- My laptop running NMEA parser + feature extractor + SENTINEL inference
- Dashboard displayed on the room projector

I also carried the u-blox F9P receiver (battery-powered) to the library entrance (Scenario A location, approximately 80 m from the room) and streamed its data back via laptop tethering.

8.2 Demonstration Narrative

At $t = 0$: receiver placed in open-sky position outside the library. Dashboard showed: all sky plot dots green, Panels 2 metrics nominal, gauge needles near-zero on P(DEGRADED).

At $t = 40$ s: I began walking the receiver toward the library entrance. The sky plot showed satellites on the south-facing azimuth sector gradually shifting from green to amber as C/N_0 started dropping.

At $t = 43$ s: CAUTION status appeared on the +5 s gauge. Panel 5 fusion bar shifted from 100 % GNSS to 82 % GNSS / 18 % IMU. The observers in Room 3058 could see the change on the projector before any visible satellite tracking change — this was the "prediction ahead of event" moment that the project is designed to demonstrate.

At $t = 48$ s: ALERT status fired on the +5 s gauge. Panel 2 C/N_0 mean dropped below the 30 dB-Hz threshold line. Two sky plot dots turned red. Panel 5 showed 35 % GNSS / 65 % IMU.

At $t = 63$ s: receiver exited the building shadow. Sky plot recovered to all-green within 4 epochs. ALERT cleared; NOMINAL restored on all three gauges within 6 epochs.

The demonstration confirmed the full system working end-to-end in real time. The 5-second early warning was visible as the CAUTION/ALERT transition on the +5 s gauge preceding the actual signal quality drop on Panel 2 by 5 epochs.

9 Architecture Comparison: Prototype vs. Production

The Dash prototype I built and the production Next.js/FastAPI system Joel built serve the same function but make different architectural trade-offs:

Table 5: Dash prototype vs. production Next.js/FastAPI stack comparison.

Aspect	**My Dash prototype**	**Joel's production stack**
Update mechanism	1 Hz polling (dcc.Interval)	WebSocket push on event
Total latency	82 ms median	<30 ms median
Figure serialisation	Full JSON each tick	Differential updates
Language	Python only	Python backend + TypeScript UI
Map library	Leaflet via Dash-Leaflet	Leaflet via React-Leaflet
Replay mode	Forward-only CSV playback	Seek-anywhere trajectory replay
Sky plot	Plotly Scatterpolar	Custom canvas renderer
SENTINEL version	Fixed checkpoint	Hot-reload model files

The Dash prototype was the right choice for rapid development in Week 2: it allowed me to build and demonstrate five working panels in four days. The WebSocket latency reduction (82 ms → <30 ms) and seek-anywhere replay mode in the production stack required an additional week of Joel's work — justified for the final demonstration, but not necessary for the validation phase.

10 Self-Reflection

10.1 Most Significant Technical Contribution

Building the **sky plot** (Panel 4) had the highest demonstrative value of any component I built. Data tables showing C/N_0 numbers tell an expert observer that signal quality is declining; the sky plot shows *why* it is declining, by making the satellite geometry and blockage pattern visually immediate.

During the live campus test, the supervisor watched three satellite dots turn from green to red in the exact azimuth sector corresponding to the library building face. His comment: "Now I understand what the EKF is fighting against." No table in the paper or equation in the slides could have produced that understanding as efficiently as 10 seconds of watching the sky plot during the demonstration.

10.2 Hardest Problem Solved

The **dcc.Store race condition** was the hardest problem I faced during the project. It was hard because the symptom (panels displaying data from slightly different epochs) was only visible during live demo, not during single-panel testing; and because my first diagnosis (a Python GIL contention issue) was wrong — the actual cause was a concurrency model specific to Dash's callback executor that I had to learn by reading the Dash architecture documentation.

The fix (single Store writer + multiple Store readers) is now a pattern I would apply from the start in any multi-panel reactive dashboard.

References

[1] P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, 2nd ed. Artech House, 2013.

[2] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[3] A. Angrisano, M. Petovello, and G. Pugliano, "Benefits of combined GPS/-GLONASS with low-cost MEMS IMUs for vehicular urban navigation," *Sensors*, vol. 12, no. 4, pp. 5134–5158, 2012.